

Employee Management System

Title: Implementation of Constructor Overloading using Employee Management System

Objective / Aim : To write a Java program that demonstrates the concept of Constructor Overloading by maintaining employee details such as name, id, designation, salary, and promotion status.

System Requirements :

Hardware:

- Standard PC/Laptop
- Intel Core i3/i5 Processor
- 4 GB or 8 GB RAM

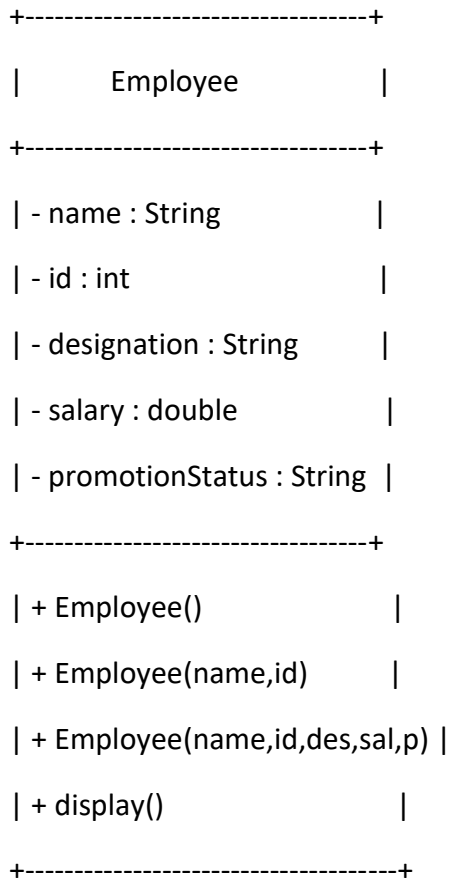
Software:

- Windows/Linux/macOS
- JDK 17 or JDK 21
- Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt or Terminal

Algorithm / Logic Description :

1. Start the program.
2. Create an Employee class with variables: name, id, designation, salary, and promotion status.
3. Create multiple constructors with different parameter lists.
4. Use constructor overloading to initialize employee details.
5. Create employee objects using different constructors.
6. Display employee details.
7. Stop the program.

UML Diagram :



Source Code :

```
class Employee {  
    String name;  
    int id;  
    String designation;  
    double salary;  
    String promotionStatus;  
    // Default Constructor  
    Employee() {  
        name = "Not Assigned";  
        id = 0;  
    }  
}
```

```
    designation = "Trainee";

    salary = 0;

    promotionStatus = "No";
}

// Constructor with two parameters
Employee(String name, int id) {
    this.name = name;

    this.id = id;

    designation = "Employee";

    salary = 25000;

    promotionStatus = "No";
}

// Constructor with all parameters
Employee(String name, int id, String designation,
    double salary, String promotionStatus) {
    this.name = name;

    this.id = id;

    this.designation = designation;

    this.salary = salary;

    this.promotionStatus = promotionStatus;
}

void display() {
    System.out.println("Name : " + name);

    System.out.println("ID : " + id);

    System.out.println("Designation : " + designation);

    System.out.println("Salary : " + salary);
}
```

```

        System.out.println("Promotion Status : " + promotionStatus);
        System.out.println();
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee(
            "Meghana", 101
        );
        Employee e3 = new Employee(
            "Ravi", 102, "Manager",
            50000, "Yes"
        );
        System.out.println("Employee 1 Details");
        e1.display();System.out.println("Employee 2 Details");
        e2.display();
        System.out.println("Employee 3 Details");
        e3.display();
    }
}

```

Compilation & Execution Commands :

Save the file as: EmployeeDemo.java

Compile: javac EmployeeDemo.java

Run: java EmployeeDemo

Output Screenshot :

```
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> javac EmployeeDemo.java
PS C:\Users\VYSHNAVI\OneDrive\Desktop\megh> java EmployeeDemo
Employee 1
Name : Not Assigned
ID : 0
Designation : Trainee
Salary : 0.0
Promotion Status : No

Employee 2
Name : Meghana
ID : 101
Designation : Developer
Salary : 30000.0
Promotion Status : No

Employee 3
Name : Mani
ID : 102
Designation : Manager
Salary : 60000.0
Promotion Status : Yes
```

Result :

Thus, the Java program to demonstrate Constructor Overloading using the Employee Management System was successfully implemented and executed.